

OmniSpace AI 全量技术文档

本地优先的全栈 AI 漫剧创作工作站

本文档面向即将加入本项目协作开发的工程师，基于代码仓库实际实现编写，覆盖系统架构、技术栈、后端与前端结构、漫剧创作管线、资源治理、数据模型、模型资产、部署运行与开发工作流。读完本文即可获得参与开发所需的完整地图。

2

常驻进程 (launcher + 后端)

19

API 路由模块

324

路由注册 (含别名)

23

SQLite 数据表

12

前端 Zustand Store

186GB

模型资产 (全量)



01 项目概览

OmniSpace AI (v2.3.1) 是一台跑在单机 Windows 工作站上的全模态创作工作站：对话、绘画、漫剧（分镜 → 资产 → 关键帧 → 视频 → 配音）、知识学习、模型管理、视频风格训练六个业务域，共用一组本地大模型与一块 12-16GB 级消费级 GPU。所有推理在本地 GPU 完成，没有云端 API 调用，数据不离开本机——这是项目的产品定位，也是全部架构决策的出发点。

1.1 核心能力矩阵

业务域	前端入口	能力	主力模型
AI 对话	/chat	多模型流式对话（WS 流式 token / reasoning）、多会话管理、附件、知识库 RAG 注入	Qwen3-VL 系列（2B/4B/8B-AWQ）、Qwen3-32B GGUF
AI 绘画	/paint	文生图 / 图生图、LoRA、ControlNet、高清修复、批量队列、实时显存遥测	SDXL base 1.0、FLUX.2-klein-4B/9B、Qwen-Image-2512
AI 漫剧	/storyboard	剧本导入、AI 分镜、A/B/C 结构化描述词、资产库（角色/场景/道具）、关键帧网格生图、逐镜视频生成与拼接、音色绑定、3D 导演台	FLUX.2-klein-4B（关键帧）、LTX-2 / Wan2.1 / CogVideoX / AnimateLCM / MiniMax H3（视频）
知识学习	/learning	文档导入向量化、RAG 检索、知识图谱可视化、浏览器代理自主上网学习、行为学习统计	bge-large-zh 嵌入 + ChromaDB
模型管理	/models	模型台账、导入校验（SHA256）、加载/卸载/热切换、显存协调	全量模型注册表
风格训练	/style	视频风格 LoRA 训练（QLoRA）、素材上传、训练轮询、版本回滚	PEFT / diffusers 训练管线
辅助能力	分散于各页	语音（Whisper ASR + TTS 装载链）、3D 图生模型（TripoSR）、图像分割（SAM）、深度估计（MiDaS）、目标检测（YOLOv8）、系统日志与流程追踪	对应各专用小模型

1.2 系统形态：三个进程、一条回环

整个系统的进程形态刻意保持极简：一个负责拉起与守护的启动器进程，一个 FastAPI 后端进程（同时也是前端的静态宿主），以及用户浏览器里的 React SPA。重模型推理按需以子进程形式存在（vLLM、ComfyUI），空闲时回收。

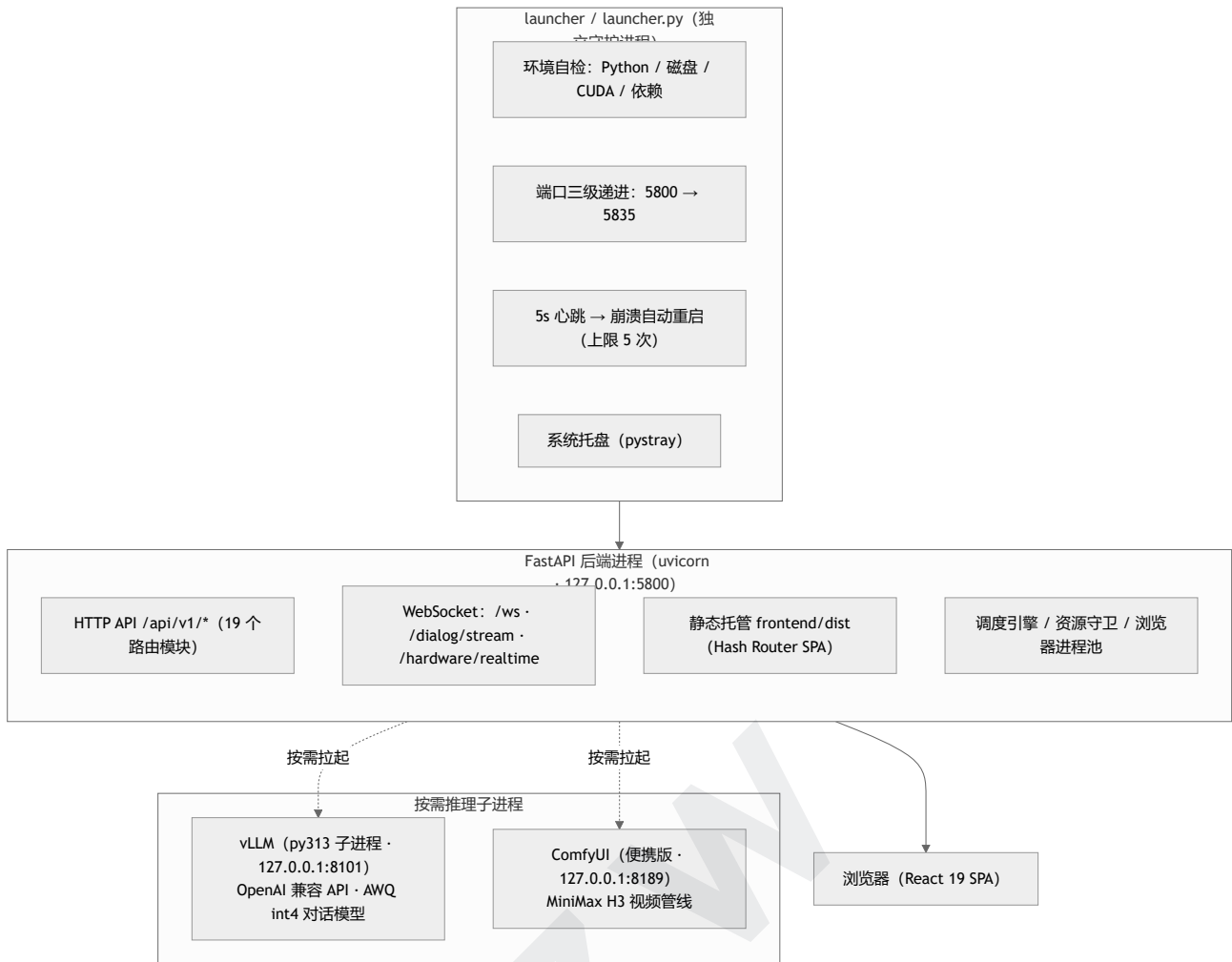


图 1: 系统进程拓扑。实线为常驻关系, 虚线为按需子进程 (空闲时回收释放显存)。

三条硬边界写在后端配置导入期, 违反任何一条服务拒绝启动:

铁律 回环绑定: `assert_loopback_host()` 在任何 socket 绑定前校验 host 必须为 `127.0.0.1 / localhost`, 否则抛 `RuntimeError` (豁免需环境变量 `OMNISPACE_ALLOW_LAN=1`)。

端口约定: 后端 `127.0.0.1:5800`, 冲突时启动器扫描 5800–5835 递进; vLLM 子进程 8101; ComfyUI 8189 (`OMNISPACE_COMFYUI_PORT` 可覆盖)。

API 前缀: 统一 `/api/v1`, 旧 `/v1` 前缀已废弃并返回 404。

1.3 代码规模与模块分布

后端为五层 Python 工程 (api / middleware / services / engines / data), 其中漫剧域因功能密度最高, 已从 4521 行单文件拆分为 9 模块的包。前端为 React 19 SPA, 9 个一级路由页、12 个 Zustand store、自研组件体系与双视觉主题族。API 层共 19 个路由文件、324 个路由注册 (含新旧路径别名), 机器枚举口径下约 270 个 HTTP 端点 + 4 个 WebSocket 端点。各模块端点分布见第 4 章。

02 技术栈全景

技术选型遵循「本地优先、消费级硬件可用、工程可维护」三个约束。下表为仓库 `package.json`、`requirements.txt` / `requirements-lock.txt` (154 包精确钉版) 与实际代码核对后的锁定清单。

2.1 前端 (frontend/)

技术	版本	用途与说明
React	19.0	Function Component + Hooks, 全部页面组件懒加载 (lazy)
TypeScript	5.6 (strict)	禁止 <code>any</code> , 运行时校验用 Zod
Vite	6.x	构建与开发服务器; <code>manualChunks</code> 分包 (three-vendor / react-vendor / vendor)
Tailwind CSS	4.x	经 <code>@tailwindcss/vite</code> 消费, <code>@theme</code> 注册双主题色阶
Zustand	5.0	状态管理, 12 个 store (漫剧域再按 6 个 slice 拆分)
React Router	7.0	<code>createHashRouter</code> Hash 路由 (静态托管 / 离线可用)
Three.js	r170	3D 导演台, 原生 API 封装 (非 React Three Fiber, 见 5.5 节说明)
Zod	4.x	运行时类型守卫
lucide-react / react-markdown / rehype-highlight / remark-gfm	—	图标、Markdown 渲染与代码高亮
vitest / ESLint 9	—	单测 (如 <code>utils/errors.test.ts</code>) 与静态检查
pnpm	11.22 (packageManager 锁定)	包管理器, 禁止 npm / yarn

2.2 后端 (backend/)

技术	版本	用途与说明
Python	3.10 (内嵌 runtime)	<code>runtime\py310\python.exe</code> 唯一解释器; vLLM 子进程独用 py313
FastAPI + Pydantic v2	≥0.115 / ≥2.10	Web 框架与数据校验, 全部函数强制 type hints
uvicorn	≥0.32	ASGI 服务器, 由 launcher 以子进程拉起
PyTorch	2.8+ (cu128)	全量推理基座; 驱动自带 CUDA Runtime, 无需单独装 Toolkit
diffusers / transformers / accelerate	≥0.31 / ≥4.46 / ≥0.29	生图、LLM、加载调度
vLLM	0.26.0+cu128 (py313 wheel)	AWQ int4 对话模型子进程推理, OpenAI 兼容 API (8101 端口)

技术	版本	用途与说明
llama.cpp (GGUF)	—	GGUF 后端 (Qwen3-32B / Codestral-22B Q4)
ComfyUI	便携版 (tools/)	独立子进程 (8189 端口) , 承载 MiniMax H3 视频管线
ChromaDB + sentence-transformers	≥0.5 / ≥2.7	向量库与 bge-large-zh 嵌入; 缺库时降级 TF-IDF
PEFT / datasets	≥0.10 / ≥2.19	LoRA / QLoRA 训练
SQLite (标准库)	WAL + FTS5	元数据存储, schema 版本 6
cryptography (AES-256-GCM) + DPAPI	≥41.0	敏感字段级加密, 密钥绑定 Windows 用户
pynvml / psutil	≥12.0 / ≥6.1	GPU / 系统资源遥测
Playwright	—	浏览器代理进程池 (知识学习自主上网)
FFmpeg	7.x (随包)	视频裁时、统一编码、拼接、静音轨补齐
loguru / ruff / pytest	—	日志、静态检查 (ruff.toml) 、测试

2.3 关键架构决策 (ADR 摘要)

A. 本地优先, 零云端依赖

所有模型推理跑在本地 GPU; 无 API Key、无数据上传。回环绑定与 TrustedHost 白名单从网络层强制这一点。

B. 单服务进程 + 静态托管

后端进程同时托管 `frontend/dist` , Hash Router 让 SPA 无需路由重写; `_ApiAwareMount` 排除 API 前缀避免静态资源覆盖接口。

C. 浏览器 + launcher 为正式交付形态

ADR-002 (2026-08-20) 裁决: 暂缓 Tauri 桌面壳, 浏览器 + 启动器为正式形态。仓库内当前无 `src-tauri/` Rust 代码。

D. 消费级硬件可用性优先

硬件基线 RTX 3060 12GB + 32GB RAM (CLAUDE.md 第 10 节) ; 16GB 显存为推荐档。所有管线都有显存不足时的降级路径 (模型路由 / 量化 / CPU 回退) 。

03 系统架构

3.1 启动时序

后端通过 `asynccontextmanager lifespan` 完成启动装配，典型耗时约 10 秒：

```
# backend/main.py lifespan 装配顺序（日志时标）
T+0s    数据库初始化（SQLite WAL）
T+3s    文件存储与缓存
T+4s    嵌入模型预加载（bge-large-zh，常驻 CUDA 低位显存段）
T+5s    调度引擎 scheduler.start()
T+5.5s  浏览器进程池预热（Chromium）
T+5.8s  资源采样器启动（30s 周期）
T+6s    WebSocket 消息中枢绑定事件循环
T+10s   就绪，launcher 心跳探测 /health 通过后打开浏览器
```

关闭阶段逆序回收：浏览器池 shutdown → WS 遥测停止 → scheduler.stop()，全程经 `event_log.log_event()` 记录 `backend_started` / `backend_stopped` 事件。

3.2 请求生命周期与中间件链

中间件按 Starlette `add_middleware` 的 insert(0) 语义注册，最后注册的在最外层。一个请求自外向内穿过：

```
请求 → TrustedHost（Host 头白名单 127.0.0.1 / localhost / [::1]，防 DNS 重绑定）
      → RequestContext（生成 request_id / duration_ms，变更类请求上报学习调度器）
      → RateLimit（滑动窗口 100 req/min/端点；/health 豁免；manga/media 独立 600/min 桶）
      → CORS（仅 localhost 任意端口，拒绝时 403 信封）
      → 异常处理器（ApiError / 校验 / HTTP / 兜底 四个，全部收敛为 200 信封）
      → 路由（19 个模块，324 个路由注册）
```

统一响应信封

所有 API 响应遵循统一信封（ADR-01，HTTP 状态恒 200，语义由信封字段承载）：

```
// 成功
{"success": true, "data": {...}, "error": null,
 "meta": {"request_id": "...", "timestamp": "ISO8601", "duration_ms": 12}}

// 失败：语义错误码 + 建议
{"success": false, "data": null,
 "error": {"code": "FEATURE_MUTEX_LOCKED", "message": "...", "detail": {...}, "suggestion": "..."},
 "meta": {...}}
```

错误码为语义串体系（`MODEL_*` / `KNOWLEDGE_*` / `BROWSER_*` / `TRAINING_*` / `STYLE_*` / `SYSTEM_*` / `HARDWARE_*` / `FEATURE_*`，约 100 个），历史数字码（40xxx-80xxx）经 `_LEGACY_CODE_MAP` 自动映射，前端只见语义串。

3.3 WebSocket 通道

端点	用途
ws://127.0.0.1:5800/ws	通用消息中枢（ws_hub.py）：任务进度 / 完成 / 失败广播、资源告警、模型切换进度等 15 种消息类型的协议翻译
ws://.../api/v1/dialog/stream/{session_id}	对话流式通道：token / reasoning / meta / error / done / status 帧协议
ws://.../api/v1/hardware/realtime	硬件实时遥测（GPU / CPU / RAM / 磁盘）
learning 进度 WS	学习会话与训练任务进度推送

前端 services/ws.ts 对全部 WS 通道实现指数退避重连（1s → 2s → 4s ... 上限 30s），未连接期间消息进入待发送队列。

3.4 功能互斥：四类重量级功能

互斥矩阵

dialog（对话）、paint（绘画）、video_gen（视频生成）、training（训练）四类重量级功能在同一时刻只允许一类运行。互斥由 middleware/feature_lock.py 的进程内异步锁落地，跨功能请求返回 FEATURE_MUTEX_LOCKED（40007）。GPU 热保护（≥90℃）期间拒绝新任务。共享小模型类别（embedding / voice / auxiliary）不受互斥影响，常驻可用。



04 后端架构详解

4.1 五层分层与依赖方向

api/	路由层	— 参数校验(Pydantic) / 信封构造 / 降级决策
middleware/	横切层	— 错误信封 / 功能互斥锁 / 限流 / CORS / 上传闸门 / 请求上下文
services/	业务层	— 领域服务 + inference/ 推理引擎 + model_manager/ 显存协调 + scheduler/ 协同调度
engines/	资源层	— gpu_backend / vram_manager (显存记账) / memory_manager (RAM LZ4 压缩) / vllm_service
data/	存储层	— database(SQLite/WAL) / file_store / cache / vector_db / fts_store / graph_store / crypto

依赖铁律 api 层只 import services 与 data, 不直接触碰 engines; 推理引擎经 model_manager 申请显存。services/offload.py 是唯一的同步推理入口 (run_blocking = asyncio.to_thread 语义收敛), API 层不允许出现 asyncio.to_thread 直调——有专门测试 (P1-06) 锁定这一边界。

4.2 API 路由模块与端点分布

路由经 importlib.import_module 动态导入并统一挂 /api/v1 前缀。下图为本仓库 19 个路由文件的路由注册数量实测 (含新旧路径别名, 未去重):

API 路由注册分布 (按路由文件)

数据来源: 对 backend/api/ 全部 19 个文件的 @router.get/post/put/delete/websocket 装饰器统计

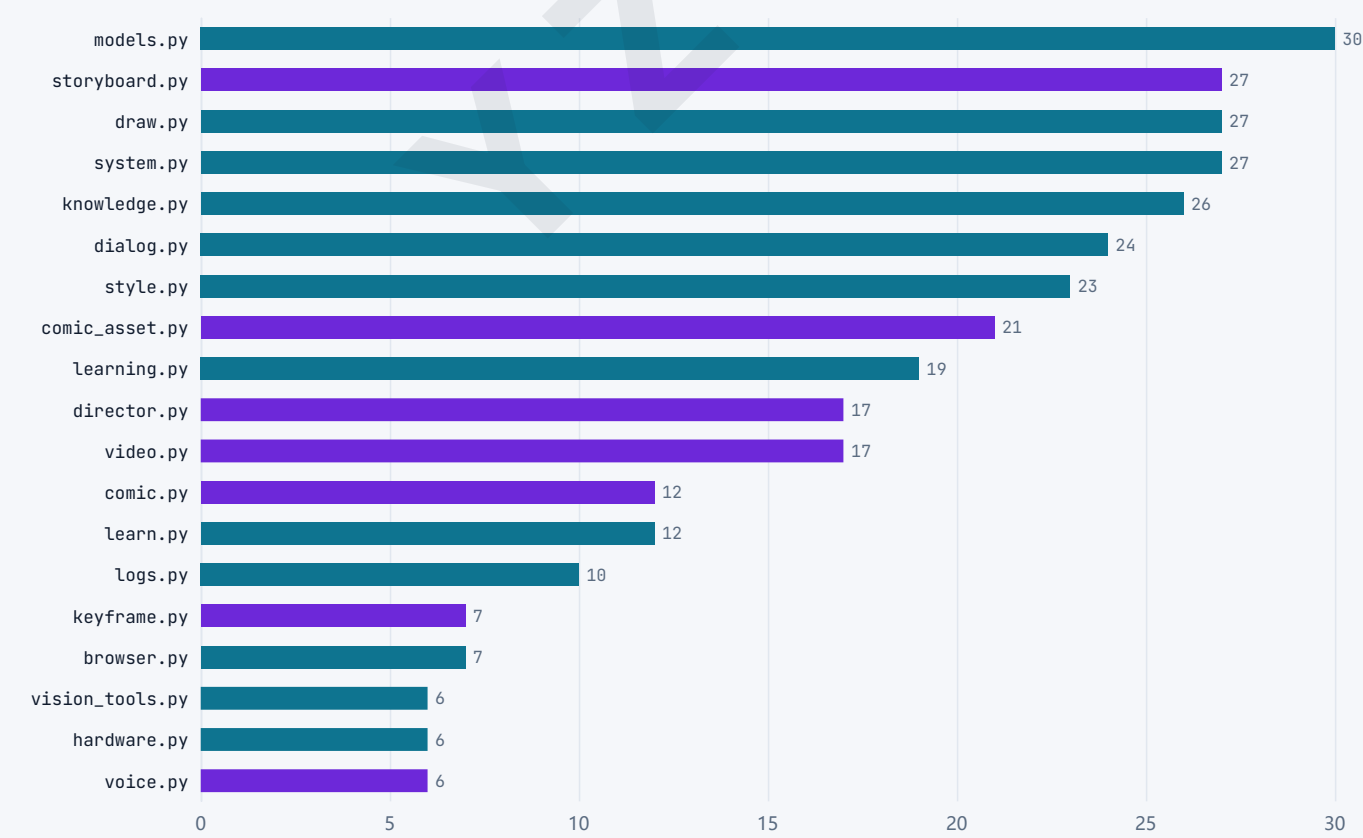


图 2: models / system / draw / storyboard / knowledge 为端点密度最高的五个模块; 漫剧域 (storyboard + comic_asset + video + director + comic + keyframe + voice) 合计约 107 个注册, 占全站三分之一。

漫剧包 (api/manga/) 九模块职责

模块	路由数	职责
common.py	0	共享层：内存态兜底存储、行/项目辅助、引擎单例、A/B/C 描述词归一化、出图规格铁律
storyboard.py	27	分镜表 CRUD、导入导出、AI 分镜与描述词生成 (<code>_ai_split_script</code>)、情绪检测
comic_asset.py	21	资产库、绑定采纳、参考图、历史与描述
video.py	17	视频生成与任务、媒体回读、叙事生成、可用模型清单
director.py	17	3D 导演台：全景图、4 合 1 截图、机位与角色、文本转 3D
comic.py	12	漫画项目 CRUD、剧本 DSL 导入、场景物件、导出包
keyframe.py	7	关键帧生成 / 批量 / 版本回滚 / 故事生图
voice.py	6	音色列表、绑定、情感、试听、上传克隆
comic_gen.py	0	生成管线层 (单图四视图 / 批量 / 重生成) , 由 comic_asset 路由调用

4.3 服务层地图 (services/)

服务	职责
领域服务 (根级)	
knowledge_service.py	知识处理管线: 过滤 → 分段 → 提取 → 质量评估 → SimHash 去重 → 向量化入库; 多级降级 (详见 6.3)
learning_scheduler.py	学习主题调度与会话管理
lora_training_service.py	LoRA / QLoRA 训练任务入队与生命周期
style_lora_service.py	视频风格 LoRA 训练
browser_agent_service.py / browser_pool.py	Playwright 浏览器代理与进程池 (自主上网学习)
behavior_service.py	用户行为日志 (训练数据素材, 字段级加密落盘)
injection_service.py	RAG 注入: 对话前知识检索与注入
event_log.py / flow_trace.py	事件日志 (30 天清理) 与 flow_id 全链路流程追踪
quality_governor.py	质量治理阈值检测 (生图每 step 轮询可中断)
priority.py / offload.py	优先级管理; 同步推理唯一入口 <code>run_blocking</code>
ws_hub.py	WebSocket 消息中枢 (内部事件 → 前端协议, 15 种消息)
资源治理 (详见第 8 章)	
resource_guard.py / resource_sampler.py	RAM/VRAM 硬限制守卫; 30s 资源采样 (环形缓冲 + JSONL 按天切分)
thermal_guard.py	GPU 温度保护链 (normal → throttling → paused 状态机)
switch_engine.py	统一模型热切换状态机 (pending → planning → unloading → loading → verifying → done, 可取消 / 回滚)
推理引擎 (inference/)	
dialog_engine.py	对话编排层: 三后端动态发现与路由 (详见 6.1)
paint_engine.py	绘画引擎: SDXL fp16 安全加载、cpu_offload、VAE slicing/tiling (详见 6.2)

服务	职责
video_engine.py	视频引擎：LTX-2(int8) / Wan2.1 / CogVideoX / AnimateLCM 多级探测链；显存不足降级 PIL Ken Burns + FFmpeg 真实 MP4（降级原因如实下发）
voice_engine.py	语音引擎：Whisper ASR + TTS 自动装载链
h3_engine.py	MiniMax H3 视频引擎，托管 ComfyUI 子进程（8189）
triposr / segment / depth / detect	3D 生成（TripoSR）、SAM 分割、MiDaS 深度、YOLOv8 检测
prompt_translator.py	中文提示词 → 英文翻译（SDXL 降级路径用）+ 工作集收缩
backends/	对话三后端：TransformersBackend（本进程直载） / GGUFBackend（llama.cpp） / VLLMBackend（子进程）
显存协调（model_manager/）与协同调度（scheduler/）	
model_manager/	ensure_loaded（全程持 _vram_lock 防并发超发）、互斥矩阵、驱逐优先级（auxiliary(0) 最先 → dialog(5) 最后）、模型缓存 / 分类 / 导入 / 验证（SHA256 + 加载测试） / 选择器（按硬件档位路由） / ML 预测器
scheduler/	analyzer（硬件分析） / decision（滞回控制决策） / dispatcher / history / monitor / video_router（视频模型路由）

4.4 引擎层与进程外服务

1. vllm_service.py

- Windows 独立 py313 子进程，OpenAI 兼容 API (127.0.0.1:8101)
- 常驻热备 + 按需冷启双策略，健康轮询 5s，启动超时 240s
- 承载 AWQ int4 对话模型（qwen3-vl-8b-awq），关闭时按需终止以释放显存

2. h3_engine.py → ComfyUI

- 托管 tools/ComfyUI_windows_portable 便携版子进程（默认 8189）
- HTTP /prompt 提交工作流，写入 ComfyUI/input/，读取 output/
- MiniMax H3 的 NVFP4 / convrot 量化 kernel 深度绑定 ComfyUI，无法迁出
- 任务完成卸载权重；彻底释放时终止子进程树

引擎全部为模块级懒加载单例（双重检查锁）。engines/vram_manager.py 负责显存分配记账（CUDA 不可用时退化为纯记账，探测失败进 _degraded_probes 如实上报）；engines/memory_manager.py 提供 RAM 块的 LRU 驱逐与 lz4 压缩。

05

前端架构详解

5.1

路由与页面

入口 `src/router.tsx` 使用 `createHashRouter` (Hash 路由适配静态托管与离线场景), 全部页面懒加载。AppShell 外壳包含 TopBar、左侧导航 (64px / 260px 折叠)、RightPanel、BottomStatusBar、全局 Toast、TechParticles 粒子层与 WarmupModal (模型冷启动进度弹窗)。

路由	页面组件	功能
/chat	components/dialog/DialogPage	AI 对话 (WS 流式、会话列表、消息气泡)
/paint	components/paint/PaintPage	AI 绘画 (参数面板、图像网格、视频面板)
/storyboard	components/manga/MangaPage	漫画创作 (库 ↔ 工作区切换, 含 3D 导演台)
/learning	components/learning/LearningPage	知识学习 (仪表盘、图谱、浏览器视图、文档导入)
/models	components/model/ModelManager	模型管理 (轻量模块, 不参与功能互斥)
/style	components/style/StylePage	视频风格 LoRA 训练
/settings	components/Settings	系统设置
/logs	components/logs/LogsPage	系统日志与流程追踪 (FlowGraph / FlowTimeline)
/help	components/help/HelpPage	帮助中心

根路径与未匹配路由均重定向到 `/chat`。

5.2

状态管理 (12 个 Zustand store)

Store	职责
useAppStore	全局: 主题 (4 主题)、字号、功能互斥 <code>activeFeature</code> 、设置、Toast
useDialogStore	会话与消息管理、WS 流式发送、生成参数 (8B/4B/2B 路由、温度、上下文长度、深度思考)
useMangaStore	漫画域入口, 按 6 个 slice 拆分: <code>projectSlice / rowsSlice / assetSlice / keyframeSlice / voiceSlice / videoSlice</code>
usePaintStore	绘画参数、生成结果、模型路由表、视频任务
useModelStore	模型列表、选择、导入、SHA256 校验、加载/卸载
useLearningStore / useLearnStore	知识学习 v2.3 (主题 CRUD、会话、设置、图谱统计) 与训练任务队列
useStyleStore	素材上传、LoRA 参数、训练轮询 (2s)、版本回滚
useHardwareStore	硬件静态画像 (GET /hardware/info) + 实时遥测 (WS) + 协同调度轮询
useTaskStore	全局任务列表 (WS Hub 广播 task_progress / complete / error) 与任务控制
useWarmupStore	对话模型冷启动弹窗 (进度条、后台继续)

5.3

API 层与代理

`services/api.ts` 以同源推导基地址 `${location.origin}/api/v1` (`file://` 场景回退 `http://127.0.0.1:5800/api/v1`) , 统一解析信封、失败抛 `ApiError` , 提供 `request / get / post / put / del / upload` 便捷方法。开发模式下 Vite 代理:

代理路径	目标	说明
<code>/api/v1</code>	<code>http://127.0.0.1:5800</code>	全部开启 <code>ws: true</code> (WebSocket 代理)。前端 dev server 运行于 5173 端口
<code>/v1</code>	<code>http://127.0.0.1:5800</code>	
<code>/ws</code>	<code>http://127.0.0.1:5800</code>	
<code>/health</code>	<code>http://127.0.0.1:5800</code>	

5.4 视觉体系：双主题族与分层 CSS

前端承载四套主题，按「家族 × 明暗」二维组织，通过 `<html data-family="tech" data-theme="light">` 双属性挂载切换：

Sakura 家族 (夜樱 · 拂晓)

主色 `#FF6B9D`。夜樱为暗色默认，拂晓为亮色。`sakura-theme.css` 注册 `sakura-*` / `mint-*` 色阶到 Tailwind 4 `@theme`。

Nebula 家族 (深空 · 晨辉)

主色 `#22D3EE`。`tech-theme.css` 对 Sakura 色阶做重映射，全站工具类整体换色，叠加 `tech-fx.css` 未来科技特效层与 TechParticles 粒子。

样式源为 `src/styles/` 下的分层体系：`tokens.css` (设计令牌唯一权威来源) → `sakura.css` (编排入口，按序引入 13 个 CSS) → 主题注册层 (`sakura-theme` / `tech-theme`) → 逐层精修 (`app` / `polish` / `deep-polish` / `aurora-motion` / `signature` / `controls` / `interactions` / `premium` / `noir` / `refinement` / `tech-fx` / `dazzle`)。所有颜色经 CSS 变量 `var(--color-*)` 引用，禁止硬编码色值。

注意 组件库为**自研体系** (`common/` 下的 `Button` / `Input` / `Modal` / `Progress` / `Slider` / `Toast` / `Tooltip` / `VirtualList` 等)，并非 `shadcn/ui`。CLAUDE.md 中的 `shadcn/ui` 与 R3F 表述属于早期规划，与当前代码存在差异 (详见 12.4 节差异对照表)。

5.5 3D 导演台

位于 `components/manga/DirectorStage.tsx`，实现方式为**原生 Three.js r170 WebGL2 封装** (非 React Three Fiber)。自研封装层位于 `src/three/`：

- `SceneManager.ts` — 场景 / 相机 / 渲染器 / 动画循环管理
- `RaycastSelector.ts` — Raycaster 对象点选
- `GizmoController.ts` — 变换控制器
- `MultiCameraRenderer.ts` — 多机位渲染 (配合 4 合 1 截图)
- `CubeCameraPanorama.ts` — 全景相机
- `SkeletonAnimator.ts` — 骨架动画

交互：左键拖拽旋转、右键平移、滚轮缩放、右键菜单 (锁定 / 解锁 / 删除 / 姿态)；远程桌面环境自动切换静态截图模式。

5.6 构建与产物

`pnpm build = tsc --noEmit && vite build`，产物输出 `frontend/dist/` (`manualChunks` 拆分 `three-vendor` / `react-vendor` / `vendor`，`sourcemap` 关闭)。生产模式下由后端进程静态托管 `dist`。

高频踩坑 前端改动**必须执行** `pnpm build` **重建 dist**, 生产模式 (launcher 启动) 才能看到变化——只改 `src/` 不构建等于没改。开发期请用 `pnpm dev` (5173 + 代理) 获得热更新。

YZW

06 核心业务模块

6.1 AI 对话 (dialog)

对话编排层 `services/inference/dialog_engine.py` 维护三个可插拔后端，按模型形态与硬件档位动态路由：

后端	实现	适用
TransformersBackend	本进程直载 (bf16 + device_map) , 预检显存不足时先腾挪绘画引擎	qwen2-vl-2b / qwen3-vl-4b 等中小模型
VLLMBackend	py313 独立子进程 (8101 端口, OpenAI 兼容 API) , PagedAttention + AWQ int4	qwen3-vl-8b-awq 高档路由, 常驻热备
GGUFBackend	llama.cpp 后端 (缺失时门控禁用)	qwen3-32b / codestral-22b Q4 量化 (16GB 卡需 CPU offload)

流式输出走 WebSocket (`/api/v1/dialog/stream/{session_id}`) , 帧协议含 token / reasoning / meta / error / done / status。上下文约束：最大上下文 8192 token、单条消息 32768 字符、预填充上限 3072 token (16GB 显存安全线)。对话前可经 `injection_service` 注入 RAG 检索结果；对话内容 AES-256-GCM 加密落盘。

6.2 AI 绘画 (paint / draw)

`services/inference/paint_engine.py` 是全站生图底座，漫剧关键帧同样复用该引擎。核心策略围绕 12~16GB 显存设计：

- **SDXL base 1.0**: fp16 安全加载 (`variant=fp16`) 、cpu_offload、VAE slicing / tiling, bf16 实占约 7GB 显存
- **FLUX.2-klein-4B**: 关键帧主力 (Qwen3 中文文本编码器, 中文提示词直出, 见第 7 章) ; **klein-9B** 为单帧高质量可选底座 (optimum-quanto float8 动态量化)
- **Qwen-Image-2512**: 20B MMDiT + Qwen2.5-VL 7B 文本编码器, unsloth Q4_K_M GGUF 流式推理
- 采样器: `euler_a` / `euler` / `dpm++_2m` / `dpm++_2m_karras` / `ddim` ; 默认 20 steps、guidance 7.5、batch 上限 4
- FLUX 不可用时降级 SDXL + `prompt_translator` 中文转英文
- 每 step 轮询质量总督 (quality_governor) , 支持生成中途取消

6.3 知识学习与 RAG (learning / knowledge)

`knowledge_service.py` 的入库管线：文档过滤 → 分段 → 实体关系提取 → 质量评估 → SimHash 去重 → 向量化入库。检索侧三路合一：ChromaDB 向量检索 (bge-large-zh 嵌入) + SQLite FTS5 全文检索 (trigram 中文三字滑窗、BM25 排序、二字词 LIKE 兜底) + 知识图谱 (`kg_entities` / `kg_edges` 两表, 查询上限 200 节点 / 400 边) 。

降级链

知识管线是全站降级设计的范本：缺 `beautifulsoup4` → 正则过滤回退；缺 `chromadb` → 内存 TF-IDF 回退；缺嵌入模型 → 哈希向量回退；未注入 LLM extractor → 规则提取回退；数据库不可用 → 内存元数据回退。每级降级如实上报，不伪造能力。知识条目有生命周期状态：青年 → 成熟 → 沉淀 → 淘汰。

浏览器代理 (`browser_pool.py` Playwright Chromium 进程池) 支撑「自主上网学习」：学习调度器按主题发起会话，行为日志 (加密) 沉淀为训练素材。 `learning_topics` / `learning_sessions` / `learning_logs` / `learning_settings` 四张学习域表由服务层自建。

6.4 模型管理 (models) 与热切换

模型台账存 SQLite `models` 表；导入器自动扫描注册并提供 SHA256 + 加载测试双重校验；选择器按硬件档位路由候选。热切换由 `switch_engine.py` 状态机驱动（pending → planning → unloading → loading → verifying → done），全程可查询进度、可取消、失败回滚——前端经 WS Hub 接收切换进度广播。

6.5 语音、3D 与视觉工具

- **语音**： `voice_engine.py` 维护 TTS 自动装载链（cosyvoice → chattts → bark → Windows SAPI5 → 静音占位 WAV），逐级诚实降级、失败只试一次；ASR 走 Whisper。GPT-SoVITS 权重在库但未接合成链（如实上报状态）。漫剧音色绑定见 7.5。
- **3D**： TripoSR 单图生成 .glb（本地权重 `models/3d/TripoSR`），经导演台预览；hunyuan3d-2.1 为画质增强候选（排期中）。
- **视觉工具**： SAM ViT-H 分割、MiDaS 深度估计、YOLOv8n 目标检测，挂载于 `vision_tools.py` 路由，支撑绘画模块的 ControlNet 条件图生成。

07 漫剧创作管线

漫剧域是项目功能密度与工程深度最高的模块，也是近期迭代的主战场。端到端数据流如下：

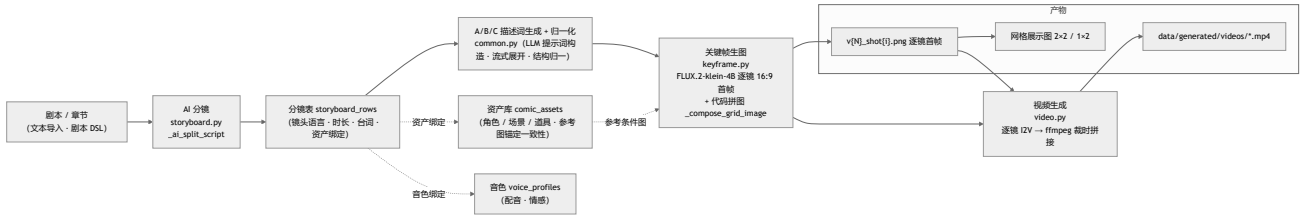


图 3：漫剧创作管线数据流。资产库通过参考条件图锚定跨镜一致性；逐镜首帧同时服务展示拼图与视频 12V。

7.1 A/B/C 结构化描述词协议

每一行分镜的描述词采用 A/B/C 三段结构，同一段文本同时驱动生图与视频生成——这是管线一致性的核心约定：

段	标题	内容与规则
A	全局风格	项目 <code>art_style</code> + 本镜剧情氛围。归一化锚点：以「A. 全局风格」标题判断描述词完整性
B	高密度世界观构建	注入绑定资产设定（角色外貌 / 场景 / 道具）保证跨镜一致；无绑定时从原文推断并如实描述。 专名必须泛化 （如「望云亭苑」→「现代沿海别墅小区」）——生图模型会按字面拆解专名
C	分镜时间轴	镜头序列，每镜格式 <code>[0.1s-1.3s]</code> 画面： <code>【标题】正文 运镜 音效</code> ；时间从 0.1s 起；头部可含「分镜网格 N×N」标记

镜头数与网格布局（恒偶数镜）

- 时长 $\geq 7s$ → **4 镜**（2×2 网格）；其余 → **2 镜**（1×2 网格）；3 镜布局已弃用（归一化时 reconcile 到 4 镜）
- 2 镜 → 1×2 网格（左右两格）；4 镜 → 2×2 田字网格，每格 1280×720 恰为 16:9 视频首帧
- 无网格标记的描述词走单帧模式（一图一镜），两种模式共存不冲突

归一化链路（common.py）

竞品外贴的原文常是单段落流式文本（段标记与时间码内联无换行）。`_expand_inline_abc` 在段标记前的句读 / 空白处与镜头时间码前插入换行完成结构化；`_normalize_direct_abc` 在关键帧生成入口归一化：A 段锚点校验、B 段自动补「高密度世界观构建：」标题、C 段时间码统一为 `[0.1s-1.3s]` 格式（容忍全角括号、空白变体、分隔符 `--`、标题前置 / 后置等抖动）、3 镜 → 4 镜 reconcile、网格标记注入。**归一化结果作为关键帧记录的 prompt 落库。**

协议铁律 未绑定资产的分行禁止生成描述词——后端在 `ai-describe` 端点进模型前校验资产绑定（须真实存在），无绑定直接拒绝；前端单行按钮禁用、批量操作跳过未绑定行并提示。生成时绑定资产的名称与 prompt 拼入 LLM 提示词，B 段强制沿用资产设定。

7.2 关键帧生成：方案 A（逐镜生成 + 代码拼图）

关键帧由 `keyframe.py` 生成，当前采用「方案 A」：含网格标记的 A/B/C 描述词 → **逐镜独立生成 16:9 首帧**（每镜 1280×720，各镜共享资产参考图锚一致性）→ `_compose_grid_image` 代码拼图（8px 黑底间隔）作为展示图。各镜首帧以 `v{N}_shot{i}.png` 命名约定单独落盘，视频侧优先读取。

模型 与提示词

- 主模型 **FLUX.2-klein-4B** (Qwen3 中文编码器, 解决 SDXL 中文图文不符)
- 逐镜提示词 `_shot_image_prompt_from_abc` = A 段风格 + B 段世界观 + C 段该镜画面正文
- 降级: FLUX 不可用 → SDXL + 中文转英文翻译
- klein-9B 保留为单帧高质量可选底座

参考条件图 (资产一致性)

- `_load_row_reference` 按角色 > 场景 > 道具优先级加载 (角色跨镜一致性最难, 场景可由泛化文本描述)
- 资产图拼接为白底横排拼图, 统一高度 640px, 最多 6 张
- 资产图像经 `asset_ids` 绑定注入

曾评估并否决的「方案 C」(换更大底座直出拼图): 网格拼图是版式控制任务, FLUX 全系 (含 Kontext) 无架构级版式优化, 换大模型不解决确定性; Kontext fp8 约 13GB 违反 12GB 硬件基线。

7.3 视频生成: 网格拆解与拼接

- 网格判据:** `_detect_row_grid` 以**关键帧记录的 prompt** (而非请求描述) 为准——用户重生成描述词后旧图仍是单帧, 按新描述误拆会得到四分之一画面; 关键帧记录不存在才退回请求描述判据; 两侧镜头数不一致回退单视频
- 首帧来源:** `_load_shot_frames` 优先读取逐镜落盘首帧 `v{N}_shot{i}.png`; 缺失才回退到拆格 (把网格图切成单镜画面)
- 逐镜分段:** `_segment_prompt` 按镜切分提示词, 逐镜 12V 生成视频段
- 裁时拼接:** `_trim_concat_segments` 用 ffmpeg 逐段裁时、统一编码 H.264 / AAC 44.1kHz 立体声后 concat; 音频缺失时 `anullsrc` 补静音轨
- 模型路由** (`backend/data/models.py` 的 VIDEO_ROUTING_TABLE): LTX-2 (int8)、Wan2.1 14B FP8 / INT4 / 1.3B、Wan2.2 TI2V 5B、LTX-Video 0.9.5、CogVideoX 2B (含 CPU 降级)、AnimateLCM (SD1.5 底座 + 运动模块, 2GB 显存档)、MiniMax H3 (ComfyUI 管线)

产物落 `data/generated/videos/`, 任务状态经 `video_tasks` 表与 WS Hub 广播 (前端 useTaskStore 消费)。

7.4 关键帧入口的功能锁

并发保护 `keyframe.py` 的 `generate / batch / story` 三个端点均持 paint 功能锁, 防止逐镜循环期间 RAM 越过 85% 线时资源守卫把在用管线当空闲模型卸载。新增任何「引擎直连 (台账外) 模型 + 守卫降载」路径时, 必须同步过 `keep_cats` 过滤 (详见 8.2 节 的事故记录)。

7.5 前端交互约定 (漫剧工作区)

- 分镜表「分镜图」列: 无图时点击「生成」打开分镜详情面板, 生图入口仅保留在详情面板的「AI 生图」主按钮
- 批量描述词弹窗默认范围「缺失生成」; 目标行为 0 时按钮禁用并提示「所有行已有描述词, 如需重新生成请切换到全部生成」
- 分镜详情面板: 描述词编辑区 9 行、4000 字上限、标注 A/B/C 段结构; 「绑定与音色」卡已移除——资产绑定归分镜表资产列管理, 音色绑定降为底部轻量入口
- 作品风格支持自定义添加, 预置风格不少于 12 种 (`art_styles` 表)

08 资源治理体系

在一块 12-16GB 显存、32GB RAM 的机器上同时服务六个业务域，资源治理是项目的工程核心。治理体系分三道闸：功能级互斥（feature_lock, API 入口拦截）→ 系统级守卫（resource_guard, 运行中兜底）→ 热保护（thermal_guard）。约束红线：**RAM $\leq$ 85%、GPU 显存 $\leq$ 90%**（充足硬件下保质缩时，不足硬件保质延时）。

8.1 resource_guard 决策流



YZW

klein-4B 级模型走 `cpu_offload` 时的内存曲线是**设计行为而非泄漏**：`from_pretrained` (bf16) 初始为 `safetensors mmap` 文件页 (uss 仅 0.6GB)；首次 `generate` 时 `offload hooks` 搬运把参数替换为私有堆副本 (klein-4B 约 16GB uss)，生成期 RAM 升高属正常；`del pipe + gc` 后张量普查 0 引用，物理页经 `EmptyWorkingSet` 归还。`free` 后私有 `commit` (vms 约 23GB) 不 `decommit`，但不占物理内存且二次加载复用不增长——可接受。排查「代码正确但行为异常」时，先查进程启动时间 vs 代码修改时间，再查 DB 残留旧数据。

YZW

09 数据存储设计

9.1 SQLite (data/omnispace.db)

元数据主库，WAL 模式、busy_timeout 5s、cache_size 16MB、NORMAL 同步、单写锁串行化（线程安全经 `threading.local`）。schema 版本 6（6 个迁移组），带 `_recover_if_corrupt()` 损坏自愈与一次性明文加密迁移（含 VACUUM 重建）。dict / list 自动 JSON 序列化。注意：无 SQLCipher 全库加密——库文件为明文，安全性依赖回环绑定 + 字段级加密 + 本机 DPAPI。

表	用途
核心业务 (16 张主表)	
dialog_sessions / dialog_messages	对话会话与消息（content 等敏感字段 AES-256-GCM 加密， <code>enc:v1:</code> 前缀自动识别解密）
projects / storyboards / storyboard_rows	漫画项目、分镜表（一项目一张）、分镜行（含描述词、镜头语言、时长、资产绑定）
comic_assets / art_styles	资产库（角色 / 场景 / 道具四视图）、自定义作品风格（预置 ≥12 种）
keyframes	关键帧（分镜行 → 生成图，多版本，prompt 存归一化后的 A/B/C 描述词——网格判据以此为准）
director_stages / director_cameras / director_characters / scene_objects	3D 导演台场景、机位、角色与对象 Transform
voice_profiles / video_tasks / train_tasks	音色配置、视频生成任务、训练任务
models / system_settings / schedule_history	模型台账、系统设置 KV、协同调度历史
功能表 (3 张)	
knowledge_fts	FTS5 全文索引（trigram 分词；不可用时内存降级）
kg_entities / kg_edges	知识图谱实体与边（实体名唯一、边去重）
学习域 (4 张，服务层自建)	
learning_topics / learning_sessions / learning_logs / learning_settings	学习主题、会话、日志（加密）、设置

合计 23 张表。CLAUDE.md 中「8 张表」的表述为早期规划，与当前 schema 不一致（见 12.4 节）。

9.2 向量库、加密与前端持久化

ChromaDB 向量库

`data/vector_db.py`，bge-large-zh 嵌入（启动时预加载常驻低位显存段）。不

字段级加密

`data/crypto.py`：AES-256-GCM，密文前缀 `enc:v1:`，密钥经 Windows

前端持久化

用户偏好走 localStorage；对话历史、绘画历史、分镜数据等大体量缓存走

可用时降级为 TF-IDF 词频余弦相似度内存存储。

DPAPI 绑定用户。覆盖
dialog_messages.content 与
behavior_logs 的 content / context /
before / after。

IndexedDB。前端禁止直接操作 SQLite
文件（一律走 API）。

生成产物不入库：图像、视频、音频、3D 模型等大文件由 `file_store.py` 按 `data/` 下的目录树管理（assets / asset_library / generated / characters / comic_assets / exports / intermediate / chroma 等），数据库只存元数据与相对路径。

YZW

10 模型资产清单

`models/` 全量约 186GB。**下载不等于可用**：模型须在引擎候选表 / 路由表中接线才生效，接线状态以 `docs/requirements-traceability.md` (M-01~M-16) 为准。磁盘占用分布：

模型资产磁盘占用 (GB)

数据来源: docs/deployment-manual.md §3 模型资产配置表 (2026-08-20 收敛口径)

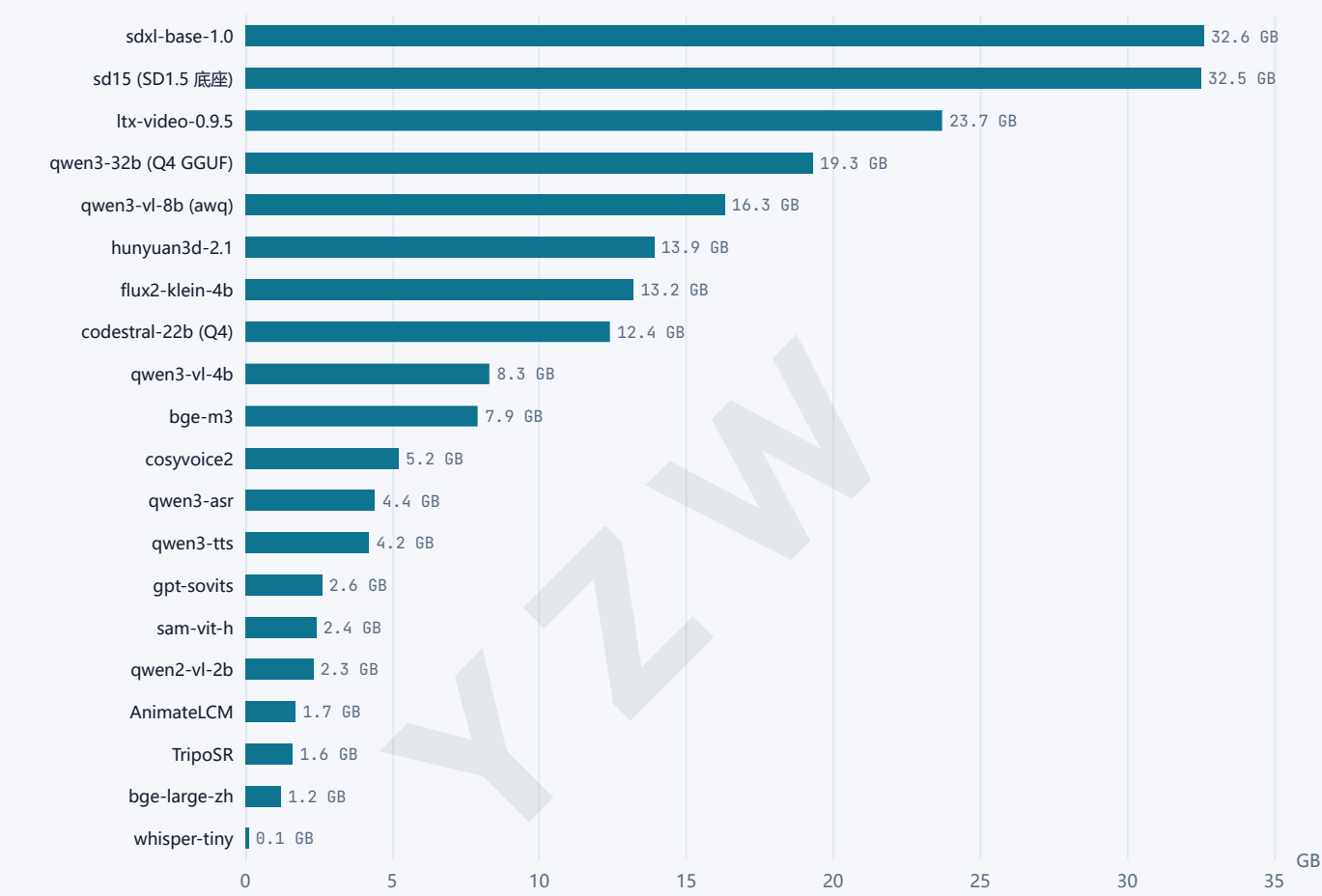


图 5: SDXL 与 SD1.5 两套绘画底座合计约 65GB, 为最大占用; 最小可跑集 (对话 + 绘画 + 视频 + 3D 四条主链路) 约 100GB。

目录	大小	用途	接线状态
paint/sdxl-base-1.0/	32.6GB	文生图主力 (SDXL base, bf16 实占约 7GB 显存)	已接线
paint/flux2-klein-4b/	13.2GB	FLUX 文生图 / 漫剧关键帧主力 (Qwen3 中文编码器)	关键帧主力
paint/flux2-klein-9b/ · 9b-fp8/	—	单帧高质量可选底座 (quanto float8 量化)	可选
paint/qwen-image-2512/	—	20B MMDiT + Qwen2.5-VL 编码器 (GGUF Q4 流式)	排期
qwen3-vl-8b/ (awq)	16.3GB	对话 / 视觉理解高档路由 (vLLM AWQ int4)	已接线
qwen3-vl-4b/	8.3GB	对话中档路由 (也是描述词生成主力)	已接线
qwen2-vl-2b/	2.3GB	对话低档路由	已接线
qwen3-32b/	19.3GB	Q4 GGUF 推理增强 (llama.cpp, 16GB 卡需 CPU offload)	已接线

目录	大小	用途	接线状态
codestral-22b/	12.4GB	Q4 GGUF 代码辅助	已接线
deepseek-r1-14b-w4a16/	—	推理模型候选	排期
video_gen/ltx-video-0.9.5/	23.7GB	视频生成主力 (LTXPipeline 动态发现; LTX-2 走 int8 量化)	已接线
video_gen/AnimateLCM/ + sd15/	1.7 + 32.5GB	img2video 兜底 (运动模块 + SD1.5 底座, 2GB 显存档; sd15 含重复副本约 22GB 清理候选)	已接线
embed/bge-large-zh/ · bge-m3/	1.2 + 7.9GB	向量嵌入 (现役) / 升级候选 (切换需重建向量库)	已接线 / 排期
3d/TripoSR/ · hunyuan3d-2.1/	1.6 + 13.9GB	单图转 3D (.glb) / 画质增强候选	已接线 / 排期
cosyvoice2/ · gpt-sovits/ · qwen3-tts/ · qwen3-asr/ · whisper-tiny/	合计约 16GB	TTS / ASR 候选链 (qwen3-tts 受 transformers 管线类缺失门控; gpt-sovits 待确认删除)	排期
sam-vit-h / depth(MiDaS) / detect(YOLOv8n)	约 2.5GB	分割 / 深度 / 检测小模型 (随包常驻候选)	已接线
lora/ · style_lora/	0.5GB	LoRA 训练产物 (训练后生成)	动态

模型路由表硬编码于 `backend/data/models.py` (含 VIDEO_ROUTING_TABLE) , 按显存档位自动路由; 无 GPU 时对话路由 CPU 档 (qwen3-vl-2b-int4-cpu / sdxl-cpu / cogvideox-cpu) 。

11 部署与运行

11.1 硬件门槛

项目	要求	说明
GPU	基线 RTX 3060 12GB ; 推荐 16GB	CLAUDE.md 硬件基线铁律: 所有功能必须在入门档位可用; 模型选型受 12GB 约束 (如否决 Kontext fp8)
内存	32GB RAM	cpu_offload 模型生成期私有副本可达 16GB+ uss
磁盘	启动门槛 $\geq 20\text{GB}$; 全量模型约 186GB (含模型 ~250GB)	launcher 自检硬门槛 <code>min_disk_space_gb=20</code>
系统	Windows 10/11 x64	加密密钥依赖 DPAPI; launcher 使用 <code>CREATE_NEW_PROCESS_GROUP</code>
CUDA	NVIDIA 驱动自带 Runtime	torch cu128 wheel, 无需单独安装 CUDA Toolkit

无 GPU / 驱动异常时系统仍可启动: 对话路由 CPU 档, 显存守门人进入降级记账模式, 降级原因如实下发。

11.2 自带运行时: 三条目录铁律

项目自带全部 Python 运行时, 不依赖系统 Python:

```
e:\OmniSpace\  
├─ runtime\py310\    内嵌 Python 3.10 (python310._pth 挂载双站点)  
├─ pydeps\          承重依赖站点 (fastapi / torch / diffusers / chromadb 等 154 包仅存于此)  
├─ backend\ frontend\ launcher\ tools\ docs\ models\ data\  
└─ runtime\py313\    vLLM 子进程专用 (0.26.0+cu128 wheel)
```

- 目录铁律**
1. `runtime/` 与 `pydeps/` 勿删勿动—— `python310._pth` 将两者挂载为双站点 (pydeps 优先), 删除 pydeps 即后端瘫痪。
 2. launcher 必须以**项目根目录**为 `cwd` 启动 (后端以 `cwd=PROJECT_ROOT` 拉起 uvicorn, 相对路径全部依赖此约定)。
 3. 解释器唯一: 任何脚本一律用 `runtime\py310\python.exe` 执行 (`.venv` 已废弃)。

11.3 启动与守护 (launcher)

`launcher/launcher.py` 是独立守护进程, 职责链: 环境自检 (OS / Python 版本 / 磁盘 $\geq 20\text{GB}$ / CUDA / 依赖关键项与可选项) → 端口管理三级递进 (首选 5800 → taskkill OmniSpace 残留进程 → 扫描 5800–5835 → 全占用抛异常) → 以子进程拉起 `uvicorn backend.main:app` (stdout/stderr 管道消费 + 日志文件) → 5s 心跳 (GET /health, 校验 `data.db = 'ok'`) → 就绪后打开浏览器与系统托盘。

CRASH_PERMANENT 后端崩溃自动重启上限 5 次, 达限进入 `crash_permanent` 放弃状态。若在 RAM 高压期 kill 后端导致 launcher 进入该状态: **恢复方法是重启 launcher 进程本身** (kill launcher PID 后以同命令重新拉起); ComfyUI (8189) 独立存活不受影响。另注意: PowerShell 查 uvicorn 进程时, 查询命令自身的包装进程命令行含 "uvicorn" 字样会自匹配造成「崩溃循环」假象, 需用 `CommandLine -like "*-m uvicorn*"` 精确过滤。

11.4 两种运行模式

```
# 生产模式（用户交付形态）：launcher 一键启动，浏览器自动打开
runtime\py310\python.exe launcher\launcher.py # → http://127.0.0.1:5800

# 开发模式：前后端分离热更新
runtime\py310\python.exe -m uvicorn backend.main:app --reload # 后端 5800
cd frontend; pnpm install; pnpm dev # 前端 5173 + API 代理

# 依赖校验 / 重建（requirements-lock.txt 为唯一真源）
runtime\py310\python.exe -c "import fastapi, torch, diffusers; print(torch.__version__, torch.cuda.is_available())"
runtime\py310\python.exe -m pip install -r requirements-lock.txt --target pydeps --no-deps
```

生产模式前端由后端静态托管（frontend/dist），无需 Node；开发模式才需要 pnpm 与 Node。前端改动需 pnpm build 重建 dist 后生产模式才可见。

11.5 常见故障速查

症状	定位与处置
生成中途模型被卸载	查功能锁是否覆盖该端点（如 keyframe 三入口持 paint 锁）；查守卫卸载日志 priority 与 0.0GB 特征（keep_cats 绕过类 bug）
模型卸载后 RAM 不回落	EmptyWorkingSet 伪句柄类问题（见 8.2 事故记录二）；确认两处修复已落地
OOM / 显存溢出	查 docs/troubleshooting.md SOOM：路由档位、量化、cpu_offload、降级链
端口冲突	launcher 自动 5800–5835 递进；手动排查残留进程（注意 uvicorn 查询假象）
DB 版本冲突	schema v6 迁移组；损坏自愈 _recover_if_corrupt()
API 直测后行为不变	ai-describe 端点只返回不落库（前端 PUT 行才落库）——先查进程启动时间 vs 代码修改时间，再查 DB 残留旧数据
改了前端没效果	生产模式必须 pnpm build；确认访问的是 dist 而非 dev server

12 协作开发指南

12.1 新协作者上手路径

Day 1 跑起来

- 按 11.4 生产模式启动，逐页过一遍六大模块
- 读 `README.md` 与本文档第 1-3 章
- 用 `pnpm dev` 起前端，改一处样式看热更新

Day 2-3 定向深潜

- 前端向：第 5 章 + `src/stores/` 任选一个 store 通读
- 后端向：第 4 章 + 挑一个路由模块对照 `docs/design/api-endpoints.md`
- AI 管线向：第 7、8 章 + `backend/api/manga/common.py`

Day 4+ 领任务

- 查 `docs/requirements-traceability.md` 找未完成项
- 前端细节 / 后端单测 / 文档润色适合起步
- 模型集成与性能优化需先读完第 8 章

12.2 代码规范精选

前端 (React 19 + TS strict)

- Function Component + Hooks, 禁 Class; 文件扩展名 `.tsx`, 组件 PascalCase, 全局组件 Omni 前缀
- 禁 `any` (用 `unknown` + 守卫 / Zod); 颜色一律 `var(--color-*)`, 禁硬编码
- `useEffect` 必须有 `cleanup`; 超 100 条列表必须虚拟化 (自研 `VirtualList`)
- AI 流式走 WS/SSE 封装, 不手写解析; 包管理 `pnpm`, 禁 `npm` / `yarn`

后端 (Python 3.10 + FastAPI)

- 全部函数写 `type hints`; API 模型一律 `Pydantic v2`; 响应走统一信封
- `asyncio` 循环中禁同步阻塞——同步推理只经 `offload.run_blocking`
- 大模型卸载三件套: `del model; torch.cuda.empty_cache(); gc.collect()`; 循环中禁重复加载模型
- 提交前 `ruff check` 通过 (`ruff.toml`); CPU 密集任务用进程池

12.3 Git 与测试

- 提交格式 `type(scope): description`, `type` 取 `feat` / `fix` / `docs` / `style` / `refactor` / `test` / `chore`, 如 `feat(chat): 实现 Qwen3-VL 多模态对话流式输出`; 分支命名 `feature/xxx`
- 测试入口唯一: `tools/run_tests.py` (定位说明见 `tests/README.md`); 后端 `pytest` (`backend/tests/`: API 冒烟、引擎纯逻辑、异步边界、host 守卫、上传守卫、遥测诚实性、vLLM 热切换压力与多模态)
- 纯函数验证用 AST 沙箱单测: `ast` 解析目标函数及模块内依赖, `exec` 到隔离命名空间 (补 `re` / `logging` / 类型注解桩), 不触发 `torch` / `FastAPI` 重型导入——适合 `common.py` 归一化逻辑这类热点
- 前端 `vitest` (`pnpm test`); E2E 流程编排见 `tests/` 与根目录 `e2e_manga_test.py`

12.4 CLAUDE.md 与代码现实差异对照 (必读)

`CLAUDE.md` 是 AI 编程助手的行为宪法 (技术栈锁定表、禁用项总表), **技术栈锁定部分仍然有效** (React 19 / Zustand 5 / `pnpm` / `FastAPI` / `SQLite` / `ChromaDB` 等), 但其中部分规划性描述已被后续裁决与代码现实覆盖。冲突时以仓库内现行文档 (`docs/INDEX.md` 登记口径) 与代码为准:

CLAUDE.md 表述	代码现实
SQLite 8 张表 (<code>chat_sessions</code> / <code>paint_history</code> ...)	实际 23 张表 (<code>dialog_sessions</code> / <code>keyframes</code> / <code>storyboard_rows</code> 等, 见第 9 章), 表名与数量均不同
3D 渲染用 React Three Fiber	导演台为原生 Three.js r170 封装 (<code>src/three/</code> 自研 <code>SceneManager</code> 等六件套)

CLAUDE.md 表述	代码现实
桌面壳层 Tauri 2.x (Rust)	ADR-002 (2026-08-20) 裁决: 浏览器 + launcher 为正式交付形态 , 仓库内无 src-tauri 代码
组件库基座 shadcn/ui	自研组件体系 (common/ 下 Button / Modal / Toast 等约 10 个)
Celery + Redis 任务队列	config.yaml 有配置项但当前未启用, 任务经进程内队列 + WS Hub 广播
CEF 120+ 浏览器内核	用户系统浏览器 + Playwright Chromium 池 (代理用)
API 端点 270+ (README 口径)	324 个路由注册 (含别名), 机器枚举口径 270 HTTP + 4 WS
硬件基线 RTX 3060 12GB + 32GB RAM	仍然有效 , 且为模型选型硬约束 (12GB 基线铁律)

12.5 文档地图 (仓库内真源)

文档	何时看
docs/design/architecture-overview.md	理解系统设计 (分层 / 中间件链 / 数据流)
docs/design/api-endpoints.md	API 对接 (270 HTTP + 4 WS 端点机器枚举总表)
docs/design/database-er.md	数据库表结构与迁移
docs/deployment-manual.md + model-deployment-plan.md	换机冷启动部署、模型资产配置 (唯一权威版)
docs/requirements-traceability.md + requirements-baseline.md	45 条需求状态唯一真源 / 需求原文基线 (领任务入口)
docs/troubleshooting.md	OOM / 端口 / DB 版本 / 模型探测四大故障
docs/design/frontend-error-policy.md	前端错误呈现三分法 (TOAST / CONSOLE / SILENT 白名单)
docs/INDEX.md	全部文档时效声明与索引 (新文档必须在此登记)
CLAUDE.md	AI 助手行为宪法 (技术栈锁定表 + 跨语言禁用项, 注意 12.4 差异)
根目录 A-E 系列 txt (仓库外源文档)	历史需求与规范源: B (开发计划, 需求可信度最高) > E (细粒度全量) > D (布局视觉) > A (语言规范) > C (技术文档); 仍有效需求已抽取入基线
docs/competitor/	竞品漫剧编辑器抓取产物 (34 文件), 漫剧模块布局对齐依据

12.6 高频踩坑清单 (实战沉淀)

坑	正确姿势
分镜图只用描述词生图	必须资产绑定注入设定, 否则同一角色跨分镜长相不一致
批量生成「没变化」假象	批量弹窗默认「缺失生成」, 已有描述词的行不会重生成; 需切换「全部生成」
4B 模型输出格式抖动	时间码 / 标题类输出必须过归一化规则 (补全标题、统一时间码格式), 正则容忍全角括号等变体
拆视频得到四分之一画面	网格判据以关键帧记录 prompt 为准, 不以请求描述为准 (用户可能重生成过描述词)
描述词 B 段出现专名	地点机构等专名必须泛化为视觉描述, 生图模型按字面拆解专名
API 直测通过但界面不变	ai-describe 只返回不落库 (前端 PUT 行才落库); 先查进程启动时间 vs 改码时间, 再查 DB 旧数据

坑	正确姿势
改前后端生产模式无变化	必须 <code>pnpm build</code> 重建 dist
RAM 高压期 kill 后端	launcher 可能 5 次重试失败进 crash_permanent; 重启 launcher 进程本身恢复
新增引擎直连模型路径	守卫降载路径必须同步过 keep_cats, 否则会被误驱逐 (8.2 事故记录一)

12.7 贡献方向建议

入门 友好

- 前端 UI 细节 (动画、交互、响应式)
- 后端 API 单元测试补充
- 文档翻译与润色

进阶

- 语音克隆与 TTS 链路优化 (qwen3-tts 接线)
- 3D 资产管线 (hunyuan3d-2.1 接线)
- 角色一致性算法提升

资深

- 新模型集成 (须过 model_manager 台账 + keep_cats)
- 资源治理与调度策略优化
- Linux / macOS 适配、多人协作创作

OmniSpace AI 全量技术文档 · 基于 v2.3.1 代码仓库实测编写 · 2026-08-26

事实来源: backend / frontend / launcher 源码与 docs/ 现行文档 (architecture-overview、api-endpoints、database-er、deployment-manual、INDEX)、CLAUDE.md、README.md; 漫剧管线与资源治理章节含 2026-08-25/26 最新工程进展。与仓库内文档冲突时, 以代码与 docs/INDEX.md 登记口径为准。

本文档位于 docs/omnispace-tech-doc/ , 自包含 HTML, 可直接发给协作者在浏览器打开。